

Car Rental Management System

Objective

You will build a console-based **Car Rental Management System** in Java. The program must allow a rental office employee to register vehicles, register customers, rent and return cars, and view reports — all through a text menu. The entire program must be written using **only** the topics listed below.

Scenario

SpeedWay Rentals is a small car rental office located near the airport. For the last eight years the office has managed everything on paper: a thick notebook for the cars, a second notebook for customers, and a stack of handwritten receipts held together by a rubber band.

The system worked — until last summer. During the holiday rush, the same car was promised to two different customers on the same morning. One of them had already paid. To make things worse, when the manager tried to calculate the monthly income, he had to add up 340 handwritten receipts by hand, and the total was wrong twice.

The owner, Mrs. Nadia, has decided that this will not happen again. She has hired you to build a simple console application that her employees can use at the front desk. She does not need a beautiful interface or a database — she needs something reliable that runs on the office computer, keeps track of which cars are available, remembers who rented what, and calculates the money correctly every single time.

She has two types of vehicles in her fleet: **regular cars** rented per day, and **luxury cars** which are also rented per day but carry an extra insurance fee and require a minimum rental period. Both share most of the same information, so she expects your design to avoid repeating yourself.

Functional Requirements

The application must allow the employee to:

1. Add a new regular car to the fleet.
2. Add a new luxury car to the fleet.

3. Add a new customer.
 4. Display all cars in the fleet.
 5. Display only the currently available cars.
 6. Rent a car to a customer for a number of days.
 7. Return a rented car.
 8. Search for a car by its ID.
 9. Search for a car by brand name.
 10. Display all customers together with the car each one currently holds.
 11. Display statistics for the office (total income, number of rented cars, most expensive car, average daily price).
 12. Exit the program safely.
-

Car

Purpose

Represents a single vehicle in the fleet. This is the **parent class** for all vehicle types. It stores the vehicle's identity, its price, and whether it is currently rented.

Fields

Each car has id, brand, model, year, price per day, available or not, car count and tax rate

Special Requirements

- All fields must be private. The outside world may only reach them through methods.
 - There must be **no setter** for id, brand, model, or year — once a car is registered these details cannot change.
 - Car count represents total number of cars.
 - Tax rate will always be 0.14
 - this must be used inside the constructors to separate parameters from fields, and inside this(...) for constructor chaining.
-

Luxury Car

Purpose

Represents a premium vehicle. A luxury car is a car, but it carries an additional insurance fee and cannot be rented for fewer than a fixed minimum number of days.

Fields

Each luxury car has insurance fee and minimum number of rental days

- insurance fee — a fixed extra amount added once to every rental of this car.
 - Minimum number of rental days — the smallest number of days a luxury car may be rented for. Which will be 3 days.
-

Customer

Purpose

Represents a person who rents cars from the office and keeps track of which car that person currently holds.

Fields

Each customer has id, name, phone, rented car id, number of rented days, total paid and count

Special Requirements

- All fields private; access through methods only.
- Count represent total number of customers.
- Rented car id represents id for the rented car, or -1 if the customer has no car.
- Total paid — the sum of everything this customer has ever paid to the office.
- Number of rented days — how many days the current rental lasts (0 when nothing is rented).

Main Class

The Main class contains the main method and controls the whole program. It must not contain business calculations that belong inside Car, LuxuryCar, or Customer — Main only reads input, calls methods, and prints results.

Required constants

Define 2 numbers:

- Max number of cars which will be 20.
- Max number of customers which will be 20.

Program flow

1. Print a welcome banner.
2. Display the menu.
3. Read the employee's choice.
4. Execute the chosen operation.
5. On exit, print a goodbye message together with the total number of cars, the total number of customers, and the total income.

Menu

```
=====
                        SPEEDWAY RENTALS SYSTEM
=====
1. Add Regular Car
2. Add Luxury Car
3. Add Customer
4. Display All Cars
5. Display Available Cars
6. Rent a Car
7. Return a Car
8. Search Car by ID
9. Search Car by Brand
10. Display All Customers
0. Exit
=====
Enter your choice:
```

Menu Options

1. Add Regular Car

Input: ID, brand, model, year, price per day.

Process: Verify the array is not full and the ID is not already used.

Output: A confirmation message with the new car's ID, or an error message explaining why the car was not added.

2. Add Luxury Car

Input: ID, brand, model, year, price per day, insurance fee.

Process: Same checks as option 1

Output: Confirmation message stating that a luxury car was added, or the reason for failure.

3. Add Customer

Input: Customer ID, name, phone number.

Process: Verify the customer array is not full and the ID is unique. Create a Customer object and store it.

Output: Confirmation with the customer's name and ID, or an error message.

4. Display All Cars

Input: None.

Output: A numbered list of every car in the fleet. If the fleet is empty, print a suitable message.

5. Display Available Cars

Input: None.

Output: The list of available cars followed by the total count. If none are available, say so clearly.

6. Rent a Car

Input: Customer ID, car ID, number of days.

Process:

1. Find the customer; if not found, stop with an error.
2. Check that the customer does not already hold a car.
3. Find the car; if not found, stop with an error.
4. Check that the car is available.
5. Check that the number of days is greater than zero.
6. If the car is a luxury car, check that the number of days is not less than the minimum rental period.
7. Calculate the cost
8. Mark the car as unavailable, record the rental in the customer object, and add the cost to the office's total income.

Output: A rental receipt showing the customer name, car brand and model, number of days, and the final cost including tax. On failure, a clear explanation of which rule was broken.

7. Return a Car

Input: Customer ID.

Process: Find the customer. If the customer holds no car, report it. Otherwise find that car in the array, mark it as available again, and clear the customer's rental record.

Output: A confirmation message naming the returned car, or an explanation if nothing was returned.

8. Search Car by ID

Input: Car ID.

Output: The full details of the matching car, or a "car not found" message.

9. Search Car by Brand

Input: Brand name.

Output: All matching cars and the number of matches, or a message stating that no car of that brand exists.

10. Display All Customers

Input: None.

Output: A list of all customers with the car each one currently holds (or "None").

0. Exit

Input: None.

Process: End the menu loop.

Output: A farewell message together with the final summary described in the Program Flow section.

Validation Rules

#	Rule
1	Car IDs must be unique. A duplicate ID is rejected.
2	Customer IDs must be unique.
3	The number of cars may never exceed max number.
4	The number of customers may never exceed max number.
5	Price per day must be greater than zero.
6	Insurance fees cannot be negative.
7	Manufacturing year must be between 1990 and 2026.
8	A car that is already rented cannot be rented again.
9	A customer may hold only one car at a time.
10	Number of rental days must be greater than zero.
11	A luxury car cannot be rented for fewer than minimum number of days.
12	A customer who has no car cannot return one.
13	Menu choices outside the listed numbers must show an “invalid choice” message and redisplay the menu.